

Internet Controlled Car

Brandon Caddow-Young

Advisor: Dr Yu-Ju Lin

April 20, 2012

Table of Contents

Introduction.....	3
Main Objectives.....	3
Background.....	4
Timeline.....	5
The Basics.....	6
Methods of Communication.....	9
Required Hardware.....	12
A Closer Look.....	13
The Range	15
Steps Involved.....	19
The Arduino.....	28
Alternative Solutions.....	33
Difficulties.....	35
Cost	37
Future Improvements.....	38
Commercial Ideas.....	40
Further Reading.....	43
Additional Information.....	44
Final Conclusion.....	45
Source Code.....	46

INTRODUCTION

This project was created as a senior project during my undergraduate degree at Charleston Southern University. My aim was to use this documentation to fully explain how each component of the project is pieced together to produce the final outcome of a small vehicle that is controlled over the internet.

MAIN OBJECTIVES

My first and foremost objective was to transform a radio frequency remote controlled car with a limited range into a radio frequency internet controlled car with an 'unlimited range'. Another objective was to enable the user to control the vehicle from a number of different devices using various different methods of communication. In order to control the vehicle from a remote location I recognized I needed to stream real time footage from the vehicle using an on-board IP camera to the user's device to enable real time steering. My degree earned in computer science at Charleston Southern has primarily been focused toward software development rather than networking. It was for this reason that I decided to choose a project that would allow myself to not only make use of my software development skills but also expand my knowledge with regards to networking principles and methodologies. I have a passion for the open source approach in general so it was my aim to build a foundation that other people could benefit from and possibly build on in the future.

BACKGROUND

The original plan was to create a vehicle that could fly or hover allowing the user could to view the ground beneath from a bird's eye view. Due to a lack of funds and time I was unable to create a flying vehicle and therefore instead resorted to using a vehicle with wheels that rather moved along the floor. This approach was less complicated although the approach used could easily be transferred to a more sophisticated vehicle.

PROJECT TIMELINE

➤ January 2012 – April 2012

- Developing a plan
- Understanding microcontrollers
- Understanding basic networking principles
 - Hypertext Transfer (or Transport) Protocol
 - External vs. Internal IP addressing
- Understanding Routers
- Understanding principles of radio frequency
- Ordering required hardware
 - Radio Frequency controlled vehicle, Arduino (microcontroller), Ethernet Shield, IP Camera, Relays, Breadboard, Battery packs, Batteries etc.

➤ May 2012 – June 2012

- Communicating with the microcontroller over the local network
- Communicating with the microcontroller over the internet
- Passing instruction from the microcontroller to the vehicle appropriately
 - Using relays to amplify signal
 - Soldering wire from the Arduino to the PCB of the transmitter
- Sending instruction to the vehicle using http requests

➤ July 2012 – August 2012

- Controlling the vehicle from a browser
- Creating a website to control the vehicle
 - Using hand gestures to control the vehicle
 - Familiarizing myself with C#
 - Understanding Mark Frosts project
 - Expanding Mark Frosts project to communicate with the vehicle
- Creating an android application to control the vehicle
 - Familiarizing myself with the Eclipse IDE
 - Learning how develop for Android Devices
 - Understanding how to use the built in accelerometer
- Installing a joysticks to be used to control the vehicle
 - Installing necessary drivers to override windows default arrow keys
- Creating a website to display step by step process for others to learn from

➤ September 2012 – December 2012

- Testing phase
- Preparing documentation and presentation
- Presenting the project

THE BASICS

In order for instruction to be successfully delivered to the vehicle the following steps should take place.

- ✓ HTTP Requests are sent from the client (user) to the server (Router of which microcontroller is physically connected to).
- ✓ The requests referred to are eventually received at the client side.
- ✓ The HTTP request is forwarded on to the Arduino (microcontroller) via its Ethernet Shield.
- ✓ The request is parsed and stripped of any 'irrelevant' information.
- ✓ The remaining 'relevant' information is used to determine what task the Arduino should perform.
- ✓ Depending on the given data, the Arduino forwards signal on to a specific connection of the transmitter.
- ✓ Radio frequency then takes care of communication between the transmitter and receiver.
- ✓ Depending on your level of understanding you may be asking yourself a couple of questions;

What is an HTTP request?

To put it simply an HTTP request is an instruction sent from one router to another router. The request can be thought of as an instruction or command, usually containing information regarding the data hoping to be received in return. For instance, to get a website pulled up on your browser your computer sent a request to another server asking for a specific page. The request looked something like this GET `http://www.brandoncaddow-young/internetCar.php` HTTP/1.1. Again, to put it in simply, you sent a message (instruction) to another server that held the contents of the website and said "I want the page internetCar.php".

What is an Arduino?

An Arduino is a microcontroller. A microcontroller is a device that generally takes input, process the data that is collected and then takes appropriate action based on a program being executed on-board the microcontroller. Microcontrollers can be thought of as mini computers capable of computation and 'add life' to a project. All electronic toys will contain some sort of a microcontroller in order to control its movement and provide the device with 'intelligence'.

Why I chose to use an Arduino

The main reason I was attracted to the Arduino was because of their open source approach to their products combined with their attitude in general. The Arduino UNO is relatively cheap although well built which makes for a great educational tool. The

Arduino team has an interesting story and some very interesting ideas of which include open source hardware.

What do you mean by open-source hardware?

Open-source hardware shares much of the principles and approach of free and open-source software. In particular, we believe that people should be able to study our hardware to understand how it works, make changes to it, and share those changes. To facilitate this, we release all of the original design files (Eagle CAD) for the Arduino hardware. These files are licensed under a Creative Commons Attribution Share-Alike license, which allows for both personal and commercial derivative works, as long as they credit Arduino and release their designs under the same license. The Arduino software is also open-source. The source code for the Java environment is released under the GPL and the C/C++ microcontroller libraries are under the LGPL.

The information above was taken directly from <http://arduino.cc/en/Main/FAQ>

METHODS OF COMMUNICATION

HTTP requests from any browser

This is the lowest level of communication offered to the user. Sending requests from a browser are inconvenient yet possible. Communication is achieved via HTTP requests which are at the core workings of this project. As demonstrated below, these HTTP requests referred to are able to be sent from any browser, anywhere in the world. All forms of communication with regards to the methods described above use HTTP requests to communicate with and control the vehicle. Here is a further explanation of how the HTTP requests are sent to the router which ultimately results in the vehicle being controlled.

`http://ROUTER IP ADDRESS/forward`

`http://ROUTER IP ADDRESS/backward`

`http://ROUTER IP ADDRESS/left`

`http://ROUTER IP ADDRESS/right`

Android Application

I developed an Android application to control the vehicle making use of the built in accelerometer. Feedback from the accelerometer is processed and an instruction is given to move the vehicle accordingly, whilst real time footage from the on-board IP

camera is streamed to the users application. The minimalist approach to the design of the application allows the user to view the streamed footage in full screen, whilst simply tilting the device to maneuver the vehicle.

Hand Movement

This section of the project was made possible by Mark Frost, a fellow Computer Science graduate of Charleston Southern university. The program was written in C# using Visual Studio making use of either a built in or 'plug and play' camera to track a specific marker around the screen. Based on the current position of the marker relative to the markers starting point, data is collected and processed before being passed on to the vehicle as instruction, eventually resulting in the vehicle being controlled according to the users movements. As for all the methods spoken of, communication is achieved via HTTP requests, this process will be further described at a later stage.

Website

I created simple yet effective website making use of simple HTML and basic JavaScript which allows for the user to control the vehicle using just the arrow keys on a keyboard. The user is also provided with the ability to move the position of the IP camera on board the vehicle by using buttons located above, below, left and right of the streamed footage. The source code is available to download by using the link below, please note

camera is streamed to the users application. The minimalist approach to the design of the application allows the user to view the streamed footage in full screen, whilst simply tilting the device to maneuver the vehicle.

Hand Movement

This section of the project was made possible by Mark Frost, a fellow Computer Science graduate of Charleston Southern university. The program was written in C# using Visual Studio making use of either a built in or 'plug and play' camera to track a specific marker around the screen. Based on the current position of the marker relative to the markers starting point, data is collected and processed before being passed on to the vehicle as instruction, eventually resulting in the vehicle being controlled according to the users movements. As for all the methods spoken of, communication is achieved via HTTP requests, this process will be further described at a later stage.

Website

I created simple yet effective website making use of simple HTML and basic JavaScript which allows for the user to control the vehicle using just the arrow keys on a keyboard. The user is also provided with the ability to move the position of the IP camera on board the vehicle by using buttons located above, below, left and right of the streamed footage. The source code is available to download by using the link below, please note

that this code belongs to Charleston Southern University and should be used for educational purposes only.

Arcade Joystick

I integrated a basic USB arcade style joystick whilst using 'already built' drivers to override the Windows arrow keys. By using this approach alongside the website I previously described made for effective, real life simulation when controlling the vehicle.

Additionally, I connected a USB SNES to PC Controller to perform a similar task, just with a little more style....

REQUIRED HARDWARE

Radio Frequency controlled vehicle

Arduino (microcontroller)

Ethernet Shield

IP Camera

Soldering Iron

Hot glue gun

Relays

Breadboard

Battery packs

Batteries

A CLOSER LOOK AT THE PROCESS

HTTP Requests are sent from the client (person controlling the vehicle) to the server (router where microcontroller physically resides).

HTTP stands for Hypertext Transfer Protocol. It's the network protocol used to deliver virtually all files and other data (collectively called resources) on the World Wide Web, whether they're HTML files, image files, query results, or anything else. Usually, HTTP takes place through TCP/IP sockets.

The information above was taken directly from <http://www.jmarshall.com/easy/http>

How does the request reach the destination router?

The HTTP requests referred to are targeted at the specific router of which the microcontroller is connected to. This is achieved by using the IP address of the router when sending the HTTP GET request. The ISP takes control of delivering the request to the appropriate router. This level of understanding is beyond this project but for more information please follow this link.

The requests referred to are eventually received at the client side.

HTTP requests are sent from a client to a server, in this projects case, delivering information to be used by the microcontroller to ultimately control the vehicle.

Throughout this project the browser acts as the client whilst the Arduino is physically connected to the server via an ethernet cable.

The HTTP request is forwarded onto the microcontroller via its Ethernet Shield.

Communication over the WAN requires both port forwarding and access to the 'internet' via an Internet Service Provider. It is vital that port forwarding is enabled in order to successfully forward the HTTP requests on from the router to the microcontroller. This step is necessary to successfully deliver the requests because whilst addressing the router by its public IP address gives no way of directly accessing the microcontroller via its internal IP address. Thus, port forwarding is used redirect any requests in the direction of the microcontroller.

The request is parsed and stripped of any 'irrelevant' information.

Once the request referred to has finally reached its destination, the microcontroller is finally able to appropriately interpret the data which has been offered. The microcontroller executes compiled source code which constantly checks for any forwarded HTTP requests from the router. This method is also known as the polling method. To put it simply, the program runs on an infinite loop, constantly checking for any new instruction.

Here is the specific code which runs on the Arduino that makes this part of the process possible.

```
void loop()
{
  Client client = server.available();
  if (client) {
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();
```

Depending on the given data, the Arduino forwards signal on to a specific connection of the transmitter

```
//FORWARD
else if(c == 'f')
{
  client.print("FORWARDS\n");
  digitalWrite(6, HIGH); //forwards
  delay(400);
  digitalWrite(6, LOW);
  delay(10);
}

//BACKWARDS
else if(c == 'b')
{
  client.print("BACKWARDS\n");
  digitalWrite(9, HIGH); //backwards
  delay(400);
  digitalWrite(9, LOW);
  delay(10);
}
```

Radio frequency then takes care of communication between the transmitter and receiver.

The fortunate part of this project is that the communication between the vehicle and the transmitter had been previously assembled which eradicated much of the hard work. I could have taken this project a step further and dealt with that communication myself although I feel it was beyond the scope of this project given my limitations with respect to time and money. With that being said, all that was required from this point on to get the car performing the correct functions was to trigger the connections on the PCB in order to successfully send appropriate signals to the car.

THE RANGE

In theory the range is unlimited as the router can be accessed from anywhere in the world through its public IP address. I say in theory because the vehicle is locally bound to the range of the radio frequency transmitter. This means that the range with respect to communicating with the radio frequency transmitter is unlimited, although locally bound by the radio frequency communication between the transmitter and the receiver on-board the vehicle. The unique and fascinating part about this project is that pieces of light are being constantly shot across the world just to cause the vehicle to make the slightest of movement. Out of interest I traced the packets involved in sending a single instruction using a tool provided by windows called TRACERT. What I learned was that the requests were being sent through numerous different countries/cities along the way just to forward on a single instruction to the vehicle.

TRACERT results for accessing this website from my computer

```

C:\Users\Brandon Caddow-Young>TRACERT www.brandoncaddow-young.com

Tracing route to www.brandoncaddow-young.com [67.220.217.235]
over a maximum of 30 hops:

  0  0 ms  0 ms  0 ms  192.168.1.1
  1  41 ms  30 ms  30 ms  host-92-21-224-1.as13285.net [92.21.224.1]
  2  47 ms  46 ms  46 ms  host-78-151-225-185.static.as13285.net [78.151.2
25.185]
  3  48 ms  45 ms  46 ms  host-78-151-225-192.static.as13285.net [78.151.2
25.192]
  4  48 ms  46 ms  45 ms  xe-11-2-0-rt001.sov.as13285.net [62.24.240.5]
  5  55 ms  46 ms  45 ms  xe-11-3-0-rt001.the.as13285.net [62.24.240.152]
  6
  7  55 ms  45 ms  46 ms  host-78-144-1-61.as13285.net [78.144.1.61]
  8  110 ms  52 ms  46 ms  host-78-144-0-145.as13285.net [78.144.0.145]
  9  63 ms  52 ms  46 ms  v209.core1.lon1.he.net [216.66.84.213]
 10 123 ms 123 ms 118 ms 10gigabitethernet7-4.core1.nyc4.he.net [72.52.92
.241]
 11 141 ms 146 ms 132 ms 10gigabitethernet8-3.core1.chi1.he.net [72.52.92
.178]
 12 164 ms 155 ms 155 ms 10gigabitethernet3-2.core1.den1.he.net [184.105.
213.86]
 13 183 ms 185 ms 187 ms 10gigabitethernet1-4.core1.lax2.he.net [72.52.92
.38]
 14 187 ms 187 ms 186 ms 216.218.213.250
 15 188 ms 186 ms 187 ms 100-42-223-154.static.webnx.com [100.42.223.154]
 16 185 ms 185 ms 187 ms 100-42-223-166.static.webnx.com [100.42.223.166]
 17 187 ms 186 ms 186 ms 67-220-217-235.hosted.static.webnx.com [67.220.2
17.235]

Trace complete.

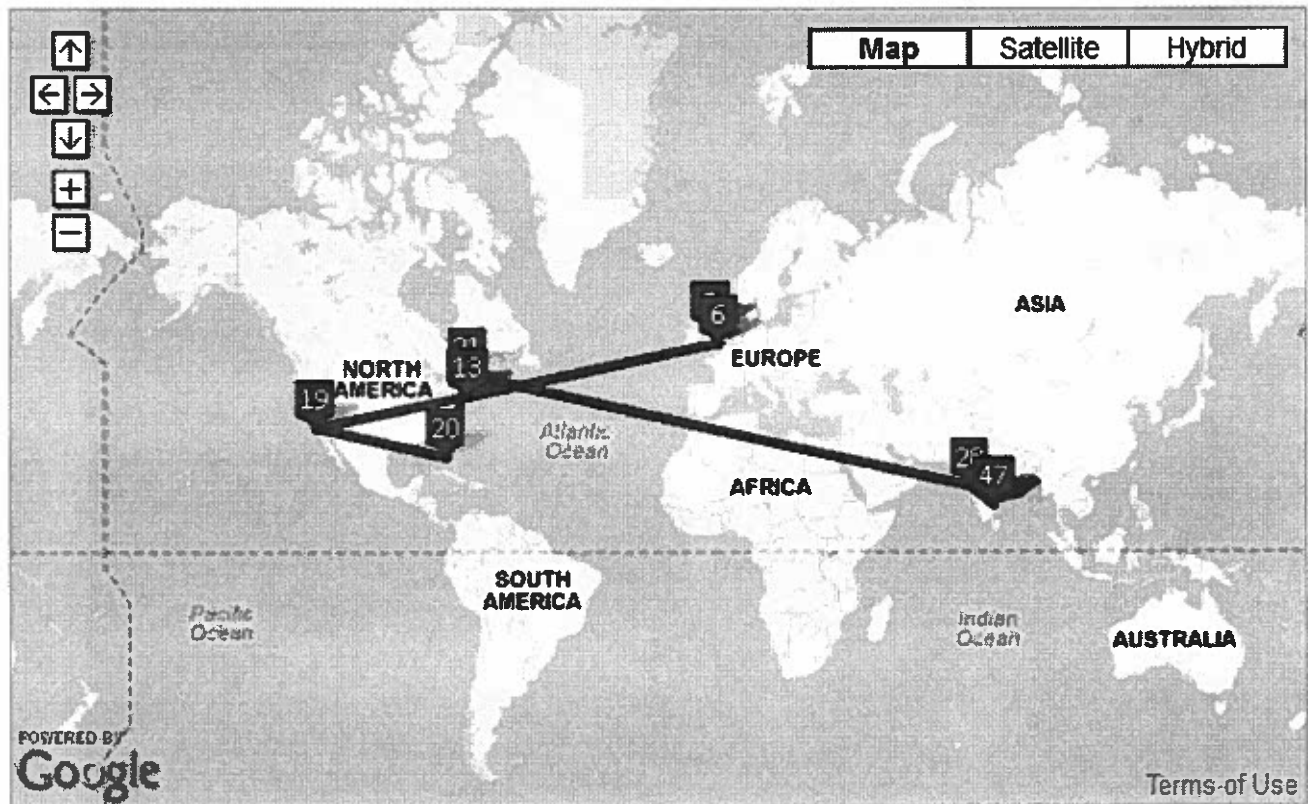
C:\Users\Brandon Caddow-Young>

```

More interestingly, I used a service provided by a website named yougetsignal.com to visually trace the route of the packets sent. With my website sitting on a server in the United States and both myself and the vehicle being located in the United Kingdom, the packets had to travel a distance of approximately 14,000 miles during its round trip! The astonishing part about the whole process is that this has to happen for each and every instruction given to the vehicle. To top it off, the the whole process takes less than a second to complete its round trip. Can you imagine the distance covered by light being shot around the world when the vehicle is driven for half an hour at a time!?

A visual representation of the route that each instruction takes

approximate geophysical trace



The two main enabling factors which allow for this extensive range are as follow; the use of the internet in general through an ISP combined with the fact that a router was used as the main method of communication. The router enables the vehicle to be controlled over both the Local Area Network and the Internet. The benefit associated with using the Local Area Network is that there is almost no lag time involved.

STEPS INVOLVED

Step 1) Buying the remote controlled car

Firstly, I purchased a relatively cheap remote controlled car for approximately \$40 from amazon.com. The scale of the vehicle was 1:10 and its maximum speed was apparently around 20mph, although I'm sure that was another marketing lie. The car is small enough to be fairly inexpensive yet big enough to successfully withstand the weight of the IP camera and the batteries involved in powering the IP camera. The battery pack included with the vehicle was a small 9.6V Ni-Cd 600mAh Rechargeable Battery Pack, that only ran for approximately five minutes, therefore I upgraded it to a Ni-Mh 3300mAh 9.6V Racing Battery Pack which lasts approximately 20 minutes.

Step 2) Buying the Arduino and the Ethernet Shield

The next step was to purchase an Arduino UNO, again from amazon.com for approximately \$30, as well as an Ethernet Shield for a further \$30. The Ethernet Shield is used as a means of communication between the router and the Arduino. As described in more detail further on, the Ethernet Shield accepts instructions from the router and forwards the instruction onto the Arduino. The Arduino takes on a similar role, taking instruction from the Ethernet Shield and forwarding the requests to the transmitter of the remote controlled vehicle.

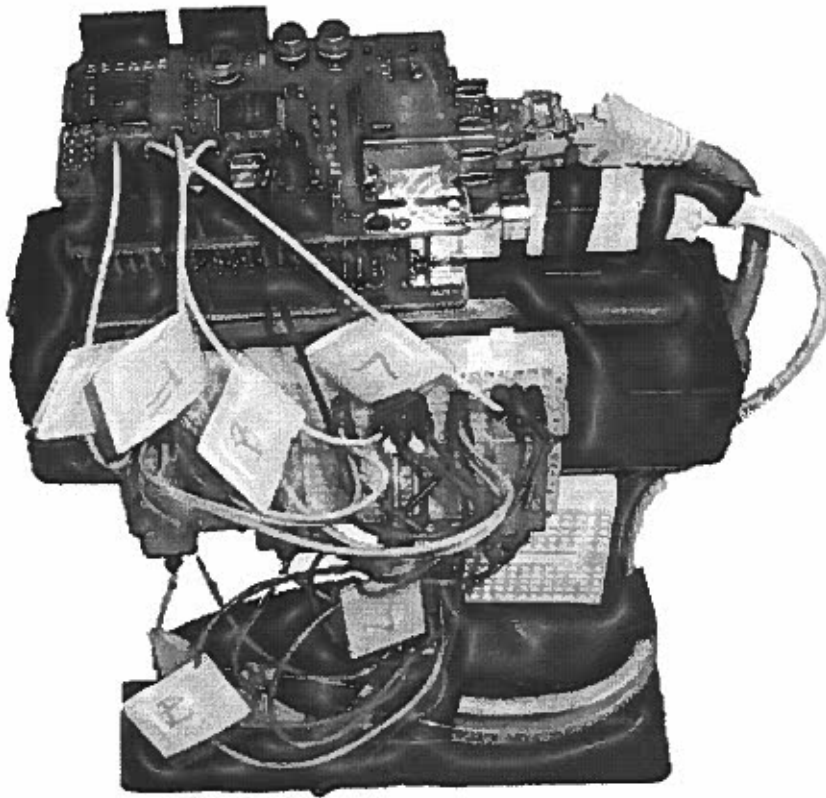
Step 3) Buying the router

A Siemens Gigaset SE587 WLAN ADSL which cost a further \$45 from amazon.com sufficiently served the purpose. The price and the range were directly correlated although my budget was the limiting factor! The router accepts instruction and acts a method of communication between a browser and the Ethernet Shield. The very design and purpose of a router combined with access to the internet via an ISP allows for universal communication, which is the very reason I chose to incorporate it into my project. Unlike the router, there was a need to create software to deal with processing of information. Therefore I created a small program which was placed on the Arduino to constantly check for activity on the router. The code takes the requests, passes the information and strips it of its 'irrelevant' information and makes use of the 'relevant' information. Based on the value of its 'relevant' information it forwards signal onto the transmitter of the vehicle.

Step 4) Setting up the router

In order for the request to successfully find the Arduino, it was essential for port forwarding to be configured. The router will have to either be assigned a static IP address (potentially expensive) or alternatively make use of DynDNS(free). Another alternative method would be to check the public IP address of the projects router, and utilize that retrieved IP address for means of communication. This method does the trick but has its drawbacks as it can be inefficient with respect to time, as it means that the public IP address would have to be checked prior to every session of communication with the vehicle. I found it also helped changing the routers configuration to assign a specific internal private IP address to the mac address of the Arduino. This way the Arduino would be forever assigned the same internal IP address by the router on reconnection. This is essential as the previously arranged port forwarding would always be directing the HTTP requests onto the IP address given on setup. This avoids lost packets/requests in the case of the router dynamically assigning a new address to the reconnected Arduino. Once the above has been configured accordingly, the job is done with respect to appropriately passing requests onto the Arduino. Once the Arduino has received these requests it is then able to process the information and take action.

Step 5) Hacking the transmitter



Regular Arduino wire was used for passing signal from the Arduino to the printed circuit board of the transmitter. Unfortunately it was not as simple as connecting the output from the Arduino to the individual connections on the PCB. In order to successfully get the transmitter to send the appropriate radio frequency signal to the vehicle I needed to find a way to trigger the specific connections on the PCB of the transmitter. The problem was that the each Arduino I/O pin outputs 40ma which was not strong enough to trigger the connection on the PCB of the transmitter. I over came the obstacle by using 40ma relays which were bought from Radio Shack. Output from the Arduino triggered one of the four relays which in turn offered the flow of ground to the PCB at a

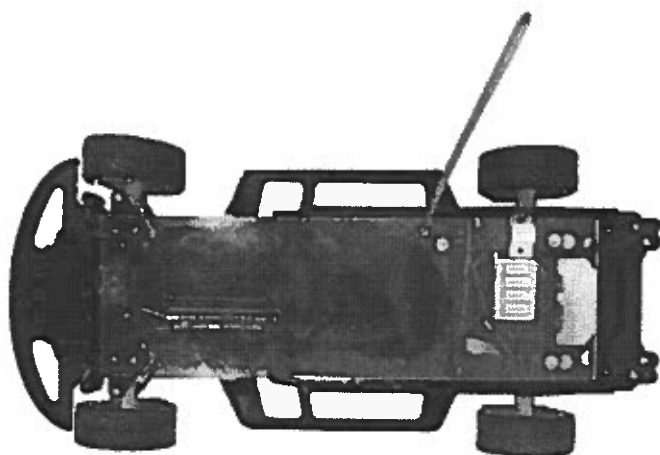
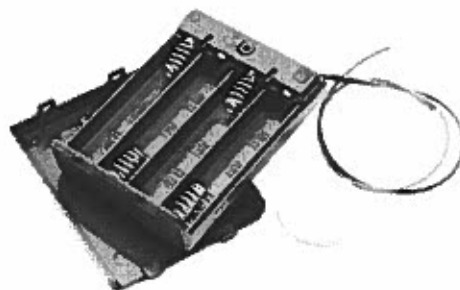
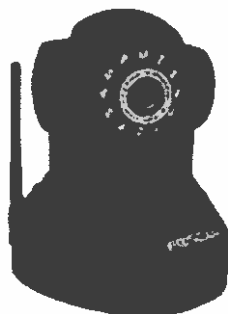
specific connection. With the board constantly powered by a 9V battery a circuit was now created, resulting in the transmitter sending appropriate signal to the receiver on-board the vehicle. In hindsight, the process was relatively simple although bringing my ideas to reality took some time and thought. The physically tricky part came in actually connecting the Arduino to the PCB of the transmitter. My soldering skills were apparently relatively poor which made for much frustration in getting each piece of wire successfully connected to the PCB. It didn't help that the connections on the specific PCB which I was trying to hack were very small relative to other connections seen on other PCB'S. In retrospect I would try choose a transmitter with a higher hack-ability rating!

Step 6) Buying the IP camera

I used a basic IP camera that cost approximately \$65 from amazon.com. Obviously the wider the range the better as my 'local range' is dependent on the distance of which IP camera is able to communicate with the router. Unfortunately my budget was again the limiting factor, by spending more money would immediately increase that 'local range' I keep referring to. The IP camera was setup designed to be accessed from any browser using its built in features. I took advantage of Foscam's work as integrated the streamed footage in a simple HTML site.

Step 7) Hacking the IP camera's power supply

Most IP cameras are powered by some sort of wiring that draws electricity from the wall, this tethered approach wasn't suitable for what I needed to accomplish. In order for the vehicle to roam around area I need an untethered alternative. Instead, I hooked the IP camera up to a small battery pack consisting of 4 AAA batteries, which was again bought from amazon.com. The four batteries outputted --- of which satisfied the input requirements of the IP camera. To ensure I did not fry the IP camera I had to ensure that the battery pack's output did not exceed the input requirements of the camera. The supply of electricity provided by the battery pack usually power's the IP camera for around approximately 30 minutes. To save on cash I invested in rechargeable batteries.



Step 8) Mounting the IP camera on the vehicle

In order for me to successfully mount the IP camera onto the vehicle I had to build a steady platform for the IP camera to rest on. I achieved this by using a piece perspex glass which was measured and cut to an appropriate size to fit neatly on top of the chassis. This allowed for a small, level step which ran from the front to that back of the vehicle. My aim was to mount the IP camera onto the car so it was discrete yet elegant. To achieve this I cut a circular whole in the plastic body of the vehicle. To accomplish this, I used Photoshop and their opacity tool. I placed one image over another, slowly altering the opacity until I got a clear indication of where I needed to do the cutting. Holes were then drilled along the indicated dotted lines and a circle was eventually removed from the body. The IP camera was then able to fit neatly into the vehicle.

Step 9) Ways of communication

Up until this point I have explained how the basis of the communication works, although in order to operate the vehicle the user would need to send HTTP requests from a browser. Although this method works, I decided to extend the project by creating different methods of communication to make the project more interesting.

Website

A simple but effective website making use of HTML and some basic JavaScript allows the user to control the vehicle using just the arrow keys on the keyboard. The user is also provided with the ability to move the position of the IP camera on board the vehicle by using buttons located above, below, left and right of the streamed footage.

IOS Application

I developed an IOS application to control the vehicle making use of the built in accelerometer. Feedback from the accelerometer sends appropriate HTTP requests resulting in the car responding to the user's request whilst real time footage from the on-board IP camera is streamed to the application. The minimalist approach to the design of the application allows the user to view the streamed footage in full screen whilst simply tilting the device to maneuver the vehicle.

Hand movement

This section of the project was created by Mark Frost, a fellow Computer Science graduate of Charleston Southern. The program is written in C# using Visual Studio making use of either a built in or plug and play camera to track a specific marker. Based on the position of the marker, the program communicates with the vehicle accordingly.

Arcade Joystick

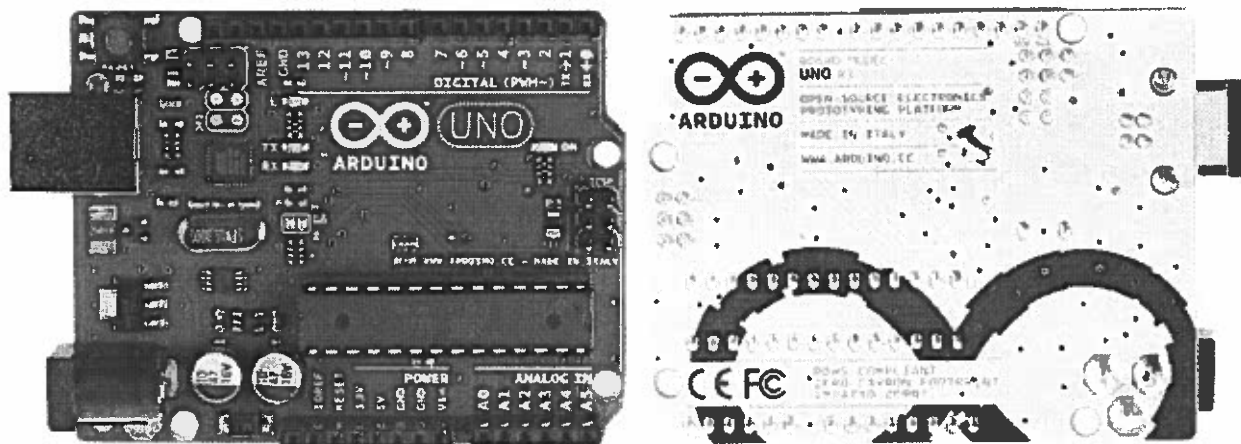
A basic arcade style joystick is connected to a computer where 'already built' drivers over ride the Windows arrow keys. By using this approach alongside the website I previously described I am able to successfully communicate with the vehicle. I also used a USB SNES to PC controller to perform a similar task.

THE ARDUINO

The Arduino UNO is a microcontroller board based on the ATmega328 (click here to download datasheet). It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz crystal oscillator, a USB connection, a power jack, an ICSP header, and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started.

The information above was taken directly from

<http://arduino.cc/en/Main/ArduinoBoardUno>



The Arduino Development Environment

Arduino provides a development environment which contains a text editor, a message area, a text console and a tool bar all combined within an easy to use graphical user interface. The software provides a form communication between the environment and

the microcontroller. More specifically this is achieved by placing the compiled code onto the microcontroller where it is executed.

Writing programs to upload to the Arduino

Programs which can be uploaded to the Arduino using the development environment are known as sketches (file extension .ino or .pde). Sketches can be written in the text editor provided. The text editor provides the ability to both cut and paste whilst also allowing the user to use other basic functions like search and replace. The editor is intelligent enough to display errors which potentially save time prior to compilation. The console referred to displays feedback offered by the Arduino environment which includes complete error messages in the case of any syntax errors.

Input and Output

The Arduino UNO provides 14 digital pins of which can be used as either input or output. Communication is achieved by using the following predefined functions offered by the Arduino environment. `pinMode()`, `digitalWrite()`, and `digitalRead()`. Each pin can provide or receive a maximum of 40 mA. This amount of electrical current was unfortunately insufficient to successfully trigger the appropriate connections on the Printed Circuit Board of the transmitter to my remote controlled car. Later on this documentation I will provide a full explanation of how I overcame this problem by using relays to provide a ground connection to the PCB.

The Arduino UNO also provides 6 analog inputs, of which each provides 10 bits of

resolution ($2^{10} = 1024$ different values). By default they measure from ground to 5 volts.

Uploading programs to the Arduino

Uploading programs to the microcontroller is fairly simple process once both the necessary drivers and the Arduino Development Environment have been installed. The user can navigate using the tool bar buttons which allow verification and uploading of the specific sketch.

Libraries

Arduino provide a number of free libraries which allow for extra functionality to the user's sketch. To take advantage of this functionality the user just has to navigate using the tool bar to Sketch > Import Library. This will ensure that the chosen library is compiled with the users sketch.

Memory

There are three approaches to storing data on the Arduino (ATmega168). As with all memory there are limitations as to how much can be stored on-board. Because of its limited capabilities the programmer is careful in the design of the program to ensure that memory is not wasted and rather used wisely.

Flash memory which is also known as program space(non-volatile).

This is where the Arduino sketch(source code) is stored. There is 32 KB bytes (0.5k is used for the boot loader) available.

SRAM also known as static random access memory(non-volatile).

This is where the sketch creates and manipulates variables at run time. There is 2 KB available. Many strings or other variable can (relatively) quickly use up this memory. To give some idea of how much memory is available each ASCII character is represented by one byte, which means that you have maximum of 1024 characters before the sketch will fail to successfully execute.

EEPROM is memory space that programmers can use to store long-term information.

There is 1 KB available.

Summary

- Microcontroller (ATmega328)
- Operating Voltage 5V
- Input Voltage (recommended) 7-12V
- Input Voltage (limits) 6-20V
- Digital I/O Pins 14 (of which 6 provide PWM output)
- Analog Input Pins 6
- DC Current per I/O Pin 40 mA
- DC Current for 3.3V Pin 50 mA
- Flash Memory 32 KB (ATmega328)
- SRAM 2 KB (ATmega328)

- EEPROM 1 KB (ATmega328)
- Speed 16 MHz

ALTERNATIVE SOLUTIONS

Other methods which could have worked

In retrospect there were a number of different approaches which I could have taken. The project would have been far cheaper with respect to both time and money, although I would not have acquired the knowledge and experience along the way.

The first alternative solution

I could have avoided using an Arduino entirely by hacking the IP camera. Similar to the method used to control the vehicle in this project, the IP camera's pan and tilt functions happen to also use HTTP requests for means of communication. The processing of the data contained within the HTTP requests has been previously configured by the manufacturer of the IP camera. So in theory, if I was to go down this route I would just be building on Foscam's work. To be more specific in my approach, this process could have been achieved by redirecting the signal from the motors which control the pan and tilt functionality to the motors which drive the wheels of the vehicle. There are extremely small IP cameras out there which offer this functionality so it is interesting to note how light and small and effortless the project could have been .

The second alternative solution

This alternative solution seems like the most feasible solution and one which I would implement if I was to redo the project. This solution involves replacing the Ethernet shield with a wifi shield. What this would mean is that communication between the microcontroller and the router would occur wirelessly. This means that the

microcontroller would be able to sit onboard the vehicle which would give the car a whole new level of intelligence. Sensors could now be easily attached to the car which could help navigate the car in several ways. Feedback from sensors onboard the vehicle could also be posted to a server which could be used through the wifi shield. The project as a whole would be less complicated since the Arduino could communicate directly with the motors on the vehicle rather than using the radio frequency transmitter and receiver.

DIFFICULTIES

Maneuvering the car so it could turn and accelerate at the same time

Unfortunately the Arduino doesn't natively support multiple threading which means that I was limited to calling only one instruction at a time. This proved to be a challenge as in order to get the car to move left or right the vehicle needed to also be accelerating or reversing at the same time. Recently I have found that there exists a library which handles 'lightweight stack-less' threads. Looking back I could have included this library and sent more than one signal at once. A short tutorial can be found by clicking [here](#).

Due to my ignorance of the library mentioned above, I chose to combat the problem from a different angle; I decided to take less efficient yet more creative path by staggering the signals sent to the vehicle. For example, when it came to both accelerating and turning at the same time I would send multiple short bursts of alternate signal to both turn the wheels and accelerate. The momentum created when sending short bursts of signal to accelerate the vehicle was enough to allow the car to move whilst the brief 'turning signals' helped steer the moving vehicle. The drawback to using this method is that it provides for a bad turning radius, especially when turning from a still position. In hindsight I would of made use of the threading library which would of allowed for numerous instructions to be executed at once.

COST

To conclude, the whole project cost me around \$150 and a whole lot of time! I am interested in taking this project forward in the near future as I feel I have created the basis of a number of possible extensions. My passion for creativity, technology and an open source approach results in me being interested in helping anyone else who is interested in continuing this project and I am willing to give away the equipment to any student who can improve on the foundation that I have created.

FUTURE IMPROVMENTS

With regards to furthering this project, here are a few possible improvements.

- Making use of the threaded library to improve turning radius.
- upgrading to a bigger, stronger, faster vehicle.
- adding distance sensors for anti crashing system.
- adding voice control.
- adding GPS for navigation purposes.
- adding a small pellet gun or some sort of device on board the vehicle.
- adding a LCD display.
- adding autonomous control.
- adding automatic charging(charging pad).

By implementing most of the above improvements would of course mean mounting an Arduino onto the vehicle itself. By doing so you are in effect giving the vehicle a brain of its own. It's intelligence being solely limited by the programmers creativity and knowledge. For instance on the account of the vehicle being driven outside of its range and left stranded, it could autonomously find its way back to a location within range of the transmitter.

How would the vehicle autonomously navigate back toward its radio frequency range?

Each instruction given to the vehicle could be pushed onto the stack. If the car was to receive not for over 20 seconds, then instructions could be popped off the stack and reversed resulting in the vehicle navigating its way back to the point where it lost

connection with the transmitter. Alternatively I could incorporate GPS by storing the x and y coordinates of each successful instruction undertaken by the RC car. If the car was to receive no instruction within 20 seconds then the car would navigate its way toward the last stored x and y coordinates, thus navigating back to a point within its radio frequency range.

Anti-crashing system

Another interesting upgrade would be to make use of distance sensors. By connecting the Arduino to the RC cars brakes would allow for the car to intelligently stop before hitting an object. If the sensor reports that there is an object within a certain distance it could either stop all instruction to the car or apply the brakes. This is something that should be implemented in every vehicle around the world!

COMMERCIAL IDEAS

With extensions of this project in mind, I foresee room for a number of possible ways in which this project could be developed into a commercial product.

Idea #1 Autonomous Lawn Mower

This idea involves an autonomous lawn mower that uses GPS to navigate its way around previously plotted x and y coordinates. An IOSX and/or ANDROID application could be developed to run on a hand held device to plot the coordinates using a birds eye view of the area of the land wishing to but cut. The application would simply allow the user to draw lines on their very own garden(as seen below) before sitting back and watching the lawn mower make its way around the garden cutting perfectly straight stripes.

My concerns regarding this idea involve the worry that I do not have the expertise to deal with the hacking of the lawn mower and would need additional mechanical support from someone with the required skills.



Idea #2 Radio Frequency Extender

This idea consists of taking my existing project, physically shrinking the overall size of the setup and turning it into a product which could be sold to the general public. The device sold would simply plug into a router via an ethernet cable. The device would consist of not only one radio frequency transmitter but many, thus allowing the user to choose the correct frequency or crystal to work with his/her own RC vehicle. This 'universal' transmitter would allow for communication between many different RC devices with an 'unlimited' range. A small program running locally could detect both the private and public IP addresses of the router in which the device was plugged into. These addresses would then be used to communicate with the remote controlled device by storing the addresses either locally or online at a predetermined location. An Iphone or Android application could then locate these addresses whilst connected to the internet and appropriately communicate with the RC device using the methods and principles previously described.

FURTHER READING

Here are a few links which route to projects similar to this one. There may be some sort of variation involved but as a whole the projects are closely related. If you are wanting to learn more about this project specifically or if there are any areas left unexplained please don't hesitate to be in contact via the comments section below.

RC Car Controlled Via the Web by Jon Bennett

<http://www.jbprojects.net/projects/wifirobot>

Building An Internet Controlled Security Robot

<http://www.shelato.com/SecurityRobot>

"Truly-Remote" Control Car Over The Internet by Joker Racer

<http://asiajin.com/blog/2009/05/20/joker-racer-truly-remote-control-car-over-the-internet/>

ADDITIONAL INFORMATION

I made every attempt to be as independent as possible throughout the duration of this project although from time to time I was guided along the way by my advisor, Dr. Yu-Ju Lin.

For a live demonstration and other information including all source code please see,
<http://brandoncaddow-young.com/internetCar.php>

FINAL CONCLUSION

Allot of time and energy was spent on this project which has resulted in myself being satisfied with the progress achieved. It is rewarding feeling to write a conclusion after spending so many hours troubleshooting and finding ways over and around obstacles. Despite my satisfaction there are many improvements (as mentioned above) that I would have loved to of implemented given fewer restrictions on time and money. This project has taught me allot with regards to networking. I now feel I have a basic understanding of some of the key concepts involved in communication over networks. Up until this point my background had been solely in software development although so it required that I stepped outside of my known comfort zone. I would like to thank Dr. Yu-Ju Lin (my advisor) for his guidance aswell as Charleston Southern University for their general support.

SOURCE CODE

Controlling the camera and the vehicle from a website

```
<html>

<head>
  <meta http-equiv="content-type" content="text/html; charset=windows-1250">
  <title>Charleston Southern University. Senior Project, 2012</title>
</head>

<style type="text/css">
  div.ex {
    position: center;
    width: 2000px;
    height: 1000px;
    border: 5px solid black;
    background-color: black;
    background-image: url('cockpit.jpg');
    background-repeat: no-repeat;
    background-position: center;
  }

  div.ex2 {
    position: center;
    width: 1000;
    height: 1000px;
    border: 5px solid red;
    background-color: black;
    background-repeat: no-repeat;
    background-position: center;
  }
</style>
<script type="text/javascript">

  var chrUser = 'admin'
  var chrPassword = ''
  var chrURL = '192.168.1.19:81' // CHANGE THIS TO COMMUNICATE WITH YOUR ROUTER

  function doCam(arg, chrType) {
    if (document.getElementById('ifrResult')) {
      if (chrType == 1) {
```

```

        chrCGI = 'decoder_control'

        } else if (chrType == 3) {
            chrCGI = 'snapshot'
        } else if (chrType == 4) {
            chrCGI = 'set_alarm'
        } else if (chrType == 9) {
            chrCGI = 'reboot'
        } else {
            chrCGI = 'camera_control'
        }
        document.getElementById('ifrResult').src = 'http://' + chrURL + '/' + chrCGI + '.cgi?user='
+ chrUser + '&pwd=' + chrPassword + '&'; + arg;
    }
}

function doCam1(arg, chrType) {
    if (document.getElementById('ifrResult1')) {
        if (chrType == 1) {
            chrCGI = 'decoder_control'

        } else if (chrType == 3) {
            chrCGI = 'snapshot'
        } else if (chrType == 4) {
            chrCGI = 'set_alarm'
        } else if (chrType == 9) {
            chrCGI = 'reboot'
        } else {
            chrCGI = 'camera_control'
        }
        document.getElementById('ifrResult1').src = 'http://' + chrURL + '/' + chrCGI + '.cgi?user='
+ chrUser + '&pwd=' + chrPassword + '&'; + arg;
    }
}
</script>

<script type="text/javascript">

function changeme(id, action) {
    if (action == "hide") {
        document.getElementById(id).style.display = "none";
    }
    else {
        document.getElementById(id).style.display = "block";
    }
}

```

```

    }
}

function changeBGC(color) {
    document.bgColor = color;
}

function Drive(x) {

    if (window.XMLHttpRequest) {
        // code for IE7+, Firefox, Chrome, Opera, Safari
        xmlhttp = new XMLHttpRequest();
    }
    else { // code for IE6, IE5
        xmlhttp = new ActiveXObject("http://192.168.1.6:80/f");
    }
    if (x == 1) {
        xmlhttp.open("GET", "http://192.168.1.6:80/f", false);
        xmlhttp.send();
    }

    if (x == 2) {
        xmlhttp.open("GET", "http://192.168.1.6:80/b", false);
        xmlhttp.send();
    }
    if (x == 3) {
        xmlhttp.open("GET", "http://192.168.1.6:80/x", false);
        xmlhttp.send();
    }

    if (x == 4) {
        xmlhttp.open("GET", "http://192.168.1.6:80/w", false);
        xmlhttp.send();
    }
}

document.onkeydown = function (evt) {
    evt = evt || window.event;
    switch (evt.keyCode) {
        case 37:
            Drive(3);
            changeme('myIMG1', 'show');
            setTimeout("Func1('myIMG1')", 1000);
            break;
        case 39:

```

```

        Drive(4);
        changeme('myIMG2', 'show');
        setTimeout("Func1('myIMG2')", 1000);
        break;
    case 38:
        Drive(1);
        changeme('myIMG3', 'show');
        setTimeout("Func1('myIMG3')", 1000);
        break;
    case 40:
        Drive(2);
        changeme('myIMG4', 'show');
        setTimeout("Func1('myIMG4')", 1000);
        break;
    }
};

```

```

function Func1(x) {
    changeme(x, 'hide');
}

```

```

function Func1Delay(x) {
    setTimeout("Func1('myIMG1')", 1000);
}

```

```
</script>
```

```
<body bgcolor="black">
```

```

    <center><div class="ex">
        <br><br><br><br>
        <a href="javascript:doCam1('command=1',1)"><font size="15" color="white">Stop
Panning</a>
        <form>

```

```

                <div align="center">
                    <center>
                        <table>
                            <tr>

```

```
width="250" height="99">
```

```
<td align="middle" valign="middle"
```

```

src="upArrow.jpg" width="128" height="99" />
<img id="myIMG3" style="display: none;"

```

```

<a
href="javascript:doCam1('command=0&onestep=5',1)"></a><br>

```

```

</tr>
</table>
</center>
</div>

```

```

<div>
<table>
<tr>

```

```

<td align="left" valign="middle" width="250" height="99">


```

```

<a href="javascript:doCam1('command=4&onestep=5',1)"></a><br>

```

```

<td width="640" height="480">

```

```

<IFRAME width="640" height="480"
src="http://192.168.1.19:81/videostream.cgi?user=admin&pwd=">

```

```

</IFRAME></td>

```

```

<td align="left" valign="middle" width="250" height="99">


```

```

<a href="javascript:doCam1('command=6&onestep=5',1)"></a><br>

```

```

</tr>
</table>
</div>

```

```

        <div align="center" ><center>
            <table>
                <tr>
                    <td align="middle" valign="middle" width="250"
height="99">
                        

                                <a
href="javascript:doCam1('command=2&onestep=5',1)"></a><br>

                                </tr>
                            </table>
                        </center>
                    </div>

                </form>
                <br><br><br><br><br><br><br>

            </div></center>

        <td></td>

        <br>

        <br>
        <iframe id="ifrResult1" name="ifrResult1" style="border: 0px; width: 1px; height:
1px;"></iframe>
        <iframe id="ifrResult" name="ifrResult" style="border: 0px; width: 1px; height:
1px;"></iframe>

    </body>
</html>

```

Arduino code

```
#include <SPI.h>
#include <Ethernet.h>

byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };

byte ip[] = {192,168,1,6}; //Local IP Address Of The Arduino

Server server(80);

void setup()
{

  Ethernet.begin(mac, ip);
  server.begin();

  pinMode(3, OUTPUT); //RIGHT
  digitalWrite(3, LOW);

  pinMode(5, OUTPUT); //LEFT
  digitalWrite(5, LOW);

  pinMode(6, OUTPUT); //FORWARDS
```

```

digitalWrite(6, LOW);

pinMode(9, OUTPUT); //BACKWARDS
digitalWrite(9, LOW);

}

void loop()
{
  Client client = server.available();
  if (client) {
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();

        if (c == '\n') {
          break;
        }

        //FOWARD
        else if(c == 'f')
        {
          client.print("FORWARDS\n");
          digitalWrite(6, HIGH); //forwards
          delay(400);
          digitalWrite(6, LOW);
          delay(10);
        }

        //BACKWARDS
        else if(c == 'b')
        {
          client.print("BACKWARDS\n");
          digitalWrite(9, HIGH); //backwards
          delay(400);
          digitalWrite(9, LOW);
          delay(10);
        }

        //LEFT
        else if(c == 'l')
        {
          client.print("LEFT\n");
          digitalWrite(5, HIGH); //left

```

```
digitalWrite(3, HIGH);//left  
delay(400);  
digitalWrite(9, LOW);  
digitalWrite(3, LOW);  
delay(10);  
}
```

```
//BACKWARDS & LEFT  
else if(c == 'z')  
{  
  client.print("BACKWARDS & LEFT\n");  
  digitalWrite(9, HIGH);//backwards  
  digitalWrite(5, HIGH);//left  
  delay(400);  
  digitalWrite(9, LOW);  
  digitalWrite(5, LOW);  
  delay(10);  
}  
}  
}
```

```
delay(1);  
client.stop();
```

```
}  
}
```

Andriod application

```
package com.example.android.accelerometerplay;

import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.client.HttpClient;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.DefaultHttpClient;
import org.apache.http.util.EntityUtils;
import android.app.Activity;
import android.content.Context;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.BitmapFactory.Options;
import android.graphics.Canvas;
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.os.PowerManager;
import android.os.PowerManager.WakeLock;
import android.util.DisplayMetrics;
import android.util.Log;
import android.view.Display;
import android.view.MotionEvent;
```

```
import android.view.Surface;
import android.view.View;
import android.view.WindowManager;
import android.widget.Button;
```

```
/**
```

AccelerometerPlayActivity is a class that inherits from Activity. It overrides the base class (Activity)'s onCreate method. Overriding occurs when you define a method in a derived class with the same signature as an existing method in the base class. This means that when onCreate is called on an instance of AccelerometerPlayActivity, it will execute the AccelerometerPlayActivity version, and not the base class (Activity)'s version.

```
*/
```

```
public class AccelerometerPlayActivity extends Activity implements
```

```
    // onClick is called when a view has been clicked.
```

```
    View.OnClickListener {
```

```
        private SimulationView mSimulationView;
        private SensorManager mSensorManager;
        private PowerManager mPowerManager;
        private WindowManager mWindowManager;
        private Display mDisplay;
        private WakeLock mWakeLock;
```

```
        public int counter = 0;
        Button forwards;
        Button backwards;
        int screenTouchx;
        int screenTouchy;
        boolean flagGo = false;
```

```
/**
```

The instruction super.onCreate(savedInstanceState) in the overridden version is explicitly calling the base class's version of the method. What this means is you want AccelerometerPlayActivity.onCreate to first execute the base class's implementation, then run some more code.

```
*/
```

```
@Override
```

```
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
        // Get an instance of the SensorManager
```

```

        mSensorManager = (SensorManager)
getSystemService(SENSOR_SERVICE);

        // Get an instance of the PowerManager
        mPowerManager = (PowerManager)
getSystemService(POWER_SERVICE);

        // Get an instance of the WindowManager
        mWindowManager = (WindowManager)
getSystemService(WINDOW_SERVICE);
        mDisplay = mWindowManager.getDefaultDisplay();

        // Create a bright wake lock
        mWakeLock =
mPowerManager.newWakeLock(PowerManager.SCREEN_BRIGHT_WAKE_LOCK,
getClass().getName());

        // instantiate our simulation view and set it as the activity's
content
        mSimulationView = new SimulationView(this);
        setContentView(mSimulationView);
    }

    /**
     * This snippet listens for touch events from the user (touch the screen, move,
     * leave the screen).
     * */
    @Override
    public boolean onTouchEvent(MotionEvent event) {

        /*
         Checking to see if the user has his thumb down or not
         If user is not touching the display then no instruction
         should be sent to the vehicle
         */
        if (event.getAction() == MotionEvent.ACTION_DOWN) {
            flagGo = true;
            // Log.d("SCREEN TOUCH:", text);
        }

        if (event.getAction() == MotionEvent.ACTION_UP) {
            flagGo = false;
            // Log.d("SCREEN TOUCH:", "LIFT");
        }
    }

```

```

        return super.onTouchEvent(event);
    }

```

```

/**

```

The foreground lifetime of an activity happens between a call to `onResume()` until a corresponding call to `onPause()`. During this time the activity is in front of all other activities and interacting with the user. An activity can

frequently go between the resumed and paused states -- for example when the device goes to sleep, when an activity result is delivered, when a new intent is delivered -- so the code in these methods should be fairly lightweight.

```

**/

```

```

@Override

```

```

    protected void onResume() {
        super.onResume();
        mWakeLock.acquire();
        mSimulationView.startSimulation();
    }

```

```

/**onPause() is where you deal with the user leaving your activity*/

```

```

@Override

```

```

    protected void onPause() {
        super.onPause();
        mSimulationView.stopSimulation();
        mWakeLock.release();
    }

```

```

class SimulationView extends View implements SensorEventListener {

```

```

    boolean globalLeft = false;
    boolean globalRight = false;
    boolean globalMiddle = false;
    boolean globalAccelerate = false;
    private Sensor mAccelerometer;
    private Bitmap left;
    private Bitmap right;
    private Bitmap go;
    private Bitmap accelerateImage;
    private Bitmap reverseImage;
    private Bitmap backwards;
    private Bitmap goLeft;
    private Bitmap goRight;
    private float mSensorX;
    private float mSensorY;

```

```

private long mSensorTimeStamp;
private long mCpuTimeStamp;

private final Driving mDriving = new Driving();

class Driving {
    private void instructCar(float sx, float sy, long timestamp) {

        // Log.d("ACCELEROMETER Y AXIS IS :",
Integer.toString((int)sy));

        /*
        ACCELERATING - DEALING WITH THE Y-AXIS
        If the Y axis value is from 0 - 6
        AND a touch(users thumb) is pressed
        AND we are not turning
        THEN accelerate!
        */
        if ((sy > -1) && (sy < 8) && (flagGo == true) && !(sx < -2) && !(sx > 2)) {
            accerlerate();
        }

        /*
        REVERSING - DEALING WITH THE X-AXIS
        if axis 10
        AND a touch(users thumb) is pressed
        AND we are not turning
        THEN reverse!
        */
        if ((sy > 9) && (flagGo == true) && !(sx < -2) && !(sx > 2)) {
            reverse();
        }

        /*
        TURNING RIGHT
        Should we accelerate forward to the right or just move right?
        */
        if (sx < -2) {
            if (flagGo == true)    AccelerateRight();
            else {
                if (flagGo == true)    turnRight();
            }

            globalRight = true;

```

```

        globalLeft = false;
        globalMiddle = false;
    }

    /*
    TURNING LEFT
    Should we accelerate forward to the left or just move left?
    */
    if (sx > 2) {
        if (flagGo == true)    AccelerateLeft();
        else {
            if (flagGo == true)    turnLeft();
        }
        globalRight = false;
        globalLeft = true;
        globalMiddle = false;
    }

    /*
    This is the middle position(with regards to turning left and right)
    Nothing should be done here
    */
    if ((sx > -2) && (sx < 2)) {
        globalRight = false;
        globalLeft = false;
        globalMiddle = true;
    }

    if (flagGo == true) globalAccelerate = true;
    else globalAccelerate = false;
    }
}

public void startSimulation() {
mSensorManager.registerListener(this, mAccelerometer, SensorManager.SENSOR_DELAY_UI);
}

public void stopSimulation() {
    mSensorManager.unregisterListener(this);
}

public SimulationView(Context context) {
    super(context);
}

```

```

        mAccelerometer =
mSensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);

        DisplayMetrics metrics = new DisplayMetrics();
getWindowManager().getDefaultDisplay().getMetrics(metrics);

        Options opts = new Options();
        opts.inDither = true;
        opts.inPreferredConfig = Bitmap.Config.RGB_565;

        left = BitmapFactory.decodeResource(getResources(),
            R.drawable.left, opts);
        right = BitmapFactory.decodeResource(getResources(),
            R.drawable.right, opts);

        go = BitmapFactory.decodeResource(getResources(),
R.drawable.go,
            opts);

        acceleratImage =
BitmapFactory.decodeResource(getResources(),
            R.drawable.accelerate, opts);
        reverselImage = BitmapFactory.decodeResource(getResources(),
            R.drawable.reverse, opts);
        backwards = BitmapFactory.decodeResource(getResources(),
            R.drawable.backwards, opts);
        goLeft = BitmapFactory.decodeResource(getResources(),
            R.drawable.goleft, opts);
        goRight = BitmapFactory.decodeResource(getResources(),
            R.drawable.goright, opts);
    }

    /**Called when sensor values have changed.*/
    public void onSensorChanged(SensorEvent event) {

        //Getting the latest data from the accelerometer
        if (event.sensor.getType() != Sensor.TYPE_ACCELEROMETER)
            return;

        switch (mDisplay.getRotation()) {
        case Surface.ROTATION_0:
            mSensorX = event.values[0];
            mSensorY = event.values[1];
            break;

```

```

        case Surface.ROTATION_90:
            mSensorX = -event.values[1];
            mSensorY = event.values[0];
            break;

        case Surface.ROTATION_180:
            mSensorX = -event.values[0];
            mSensorY = -event.values[1];
            break;

        case Surface.ROTATION_270:
            mSensorX = event.values[1];
            mSensorY = -event.values[0];
            break;
    }
}

/**
This method deals with drawing on the canvas
This method is called constantly!
*/

@Override
protected void onDraw(Canvas canvas) {

    //Drawing the appropriate images onto the display
    if (globalRight == true) {
        canvas.drawBitmap(right, 500, 600, null);
        if (globalAccelerate == true)
            canvas.drawBitmap(goRight, 250, 270, null);
    }

    if (globalLeft == true) {
        canvas.drawBitmap(left, 0, 600, null);
        if (globalAccelerate == true)
            canvas.drawBitmap(goLeft, 250, 270, null);
    }

    if ((mSensorY > -1) && (mSensorY < 8)) { // if Y axis FROM 0

        canvas.drawBitmap(go, 250, 400, null);
        if (globalAccelerate == true) {

```

```

        canvas.drawBitmap(accelerateImage, 250,
200, null);
    }
}

if ((mSensorY > 9)) { // if Y axis is greater than 9
    canvas.drawBitmap(backwards, 250, 400, null);
    if (globalAccelerate == true) {
        canvas.drawBitmap(reverseImage, 250,
200, null);
    }
}

//Getting latest data from the accelerometer
final Driving Driving = mDriving;
final long now = mSensorTimeStamp + (System.nanoTime()
- mCpuTimeStamp);

final float sx = mSensorX;
final float sy = mSensorY;

//Giving latest data from the accelerometer to my function
Driving.instructCar(sx, sy, now);
invalidate();
}

int countAccelerateLeft = 0;

private void AccelerateLeft() {
    // SEND RIGHT TO CAR USING HTTP;
    Log.d("ACCELERATING LEFT:: ", "ACCELERATING LEFT");
    countAccelerateLeft++;
    if (countAccelerateLeft % 50 == 0) {

        try {
            HttpClient client = new DefaultHttpClient();
            String getURL = "http://192.168.1.6/x";
            HttpGet get = new HttpGet(getURL);
            HttpResponse responseGet = client.execute(get);
            HttpEntity resEntityGet = responseGet.getEntity();
            if (resEntityGet != null) {
                // do something with the response
                String response =
EntityUtils.toString(resEntityGet);

                Log.i("GET RESPONSE", response);

```

```

    }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

int countAccelerateRight = 0;

private void AccelerateRight() {
    // SEND RIGHT TO CAR USING HTTP;
    Log.d("ACCELERATING RIGHT: ", "ACCELERATING RIGHT:");
    countAccelerateRight++;
    if (countAccelerateRight % 50 == 0) {

        try {
            HttpClient client = new DefaultHttpClient();
            String getURL = "http://192.168.1.6/w";
            HttpGet get = new HttpGet(getURL);
            HttpResponse responseGet = client.execute(get);
            HttpEntity resEntityGet = responseGet.getEntity();
            if (resEntityGet != null) {
                // do something with the response
                String response =
EntityUtils.toString(resEntityGet);

                Log.i("GET RESPONSE", response);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

int countLeft = 0;

private void turnLeft() {
    // SEND RIGHT TO CAR USING HTTP;
    Log.d("TURNING LEFT", "TURNING LEFT");
    countLeft++;
    if (countLeft % 100 == 0) {

        try {
            HttpClient client = new DefaultHttpClient();
            String getURL = "http://192.168.1.6/l";

```

```

        HttpGet get = new HttpGet(getURL);
        HttpResponse responseGet = client.execute(get);
        HttpEntity resEntityGet = responseGet.getEntity();
        if (resEntityGet != null) {
            // do something with the response
            String response =
EntityUtils.toString(resEntityGet);
            Log.i("GET RESPONSE", response);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

int countRight = 0;

private void turnRight() {
    // SEND RIGHT TO CAR USING HTTP;
    Log.d("TURNING RIGHT", "TURNING RIGHT");
    countRight++;
    if (countRight % 50 == 0) {
        try {
            HttpClient client = new DefaultHttpClient();
            String getURL = "http://192.168.1.6/r";
            HttpGet get = new HttpGet(getURL);
            HttpResponse responseGet = client.execute(get);
            HttpEntity resEntityGet = responseGet.getEntity();
            if (resEntityGet != null) {
                // do something with the response
                String response =
EntityUtils.toString(resEntityGet);
                Log.i("GET RESPONSE", response);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

int countAccelerate = 0;

private void accerlerate() {

```

```

// SEND ACCELERATE TO CAR USING HTTP;
Log.d("Accerating: ", "ACCELERATE!!");
countAccelerate++;
if (countAccelerate % 50 == 0) {

    try {
        HttpClient client = new DefaultHttpClient();
        String getURL = "http://192.168.1.6/f";
        HttpGet get = new HttpGet(getURL);
        HttpResponse responseGet = client.execute(get);
        HttpEntity resEntityGet = responseGet.getEntity();
        if (resEntityGet != null) {
            // do something with the response
            String response =
EntityUtils.toString(resEntityGet);

            Log.i("GET RESPONSE", response);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

int countReverse = 0;

private void reverse() {
    // SEND REVERSE TO CAR USING HTTP;
    Log.d("REVERSING!: ", "REVERSE!!");
    countReverse++;
    if (countReverse % 50 == 0) {

        try {
            HttpClient client = new DefaultHttpClient();
            String getURL = "http://192.168.1.6/b";
            HttpGet get = new HttpGet(getURL);
            HttpResponse responseGet = client.execute(get);
            HttpEntity resEntityGet = responseGet.getEntity();
            if (resEntityGet != null) {
                // do something with the response
                String response =
EntityUtils.toString(resEntityGet);

                Log.i("GET RESPONSE", response);
            }
        } catch (Exception e) {

```

```
        e.printStackTrace();
    }
}

// Add reverse left and right here

//Called when the accuracy of a sensor has changed.
public void onAccuracyChanged(Sensor sensor, int accuracy){
}

//Called when a view has been clicked.
public void onClick(View v) {
    // TODO Auto-generated method stub
}
}
```